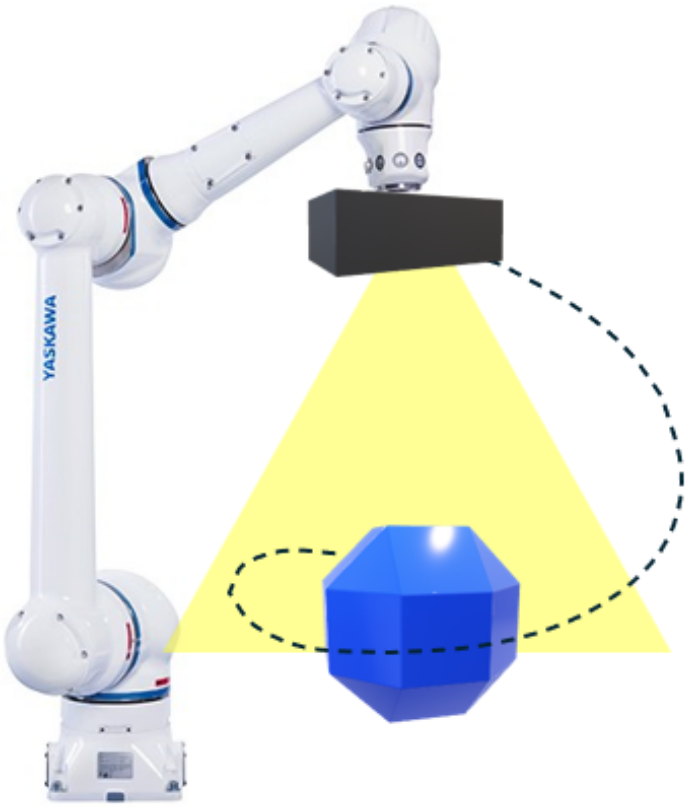
Rapport :

Intégration robotique



Systèmes Robotiques et Interactifs (SRI)

University Paul Sabatier - UPSSITECH

Rapport :

Intégration robotique

Systèmes Robotiques et Interactifs (SRI)

University Paul Sabatier - UPSSITECH

Tuteurs de projet :

- Thierry Germa

Chefs de projet :

- Mouléro Begbegni

Chefs techniques de projet :

- Axel Niato

Membres de l'equipe :

- Souha Ghazouani
- Amal Ferhani
- Nour Ghiloufi
- Rahim Saidi
- Nahaclan Coulibaly
- Loïc Maignan
- Kyllian Aloy
- Mamadou Traore
- Nelson Sanchez

Abstract

Ce projet, mené dans le cadre de notre formation à l'UPSSITECH, a pour but de nous initier à l'utilisation de ROS dans le domaine de la robotique industrielle. Il s'agit d'un projet d'intégration robotique, dont l'objectif est de développer une trajectoire pour le robot manipulateur Yaskawa HC10 afin de numériser un objet à l'aide d'un capteur 3D. Réalisé en autonomie et en groupe, chaque étape du projet est soigneusement documentée afin d'assurer la reproductibilité des résultats.

Les principales missions incluent l'apprentissage de l'environnement ROS et de l'outil MoveIt pour la génération des trajectoires, l'intégration d'un capteur 3D dans un environnement de simulation, ainsi que l'ajustement des mouvements du robot en fonction des données collectées.

Ce rapport décrit en détail le processus suivi, depuis l'analyse des besoins jusqu'à la réalisation des différentes phases et la présentation des résultats obtenus.

Table des matières

Abstract **2**

Introduction **6**

0.1 Objet du document 6

0.2 Documents applicables et de référence 7

1 Contexte **8**

1.1 Compréhension du besoin 8

1.2 Analyse du besoin 8

1.2.1 Création de la Simulation du Robot : 8

1.2.2 Ajout et intégration du capteur 3D 9

1.2.3 Construction et Génération de Trajectoire 9

1.3 Organisation 10

1.3.1 WBS - Working Breakdown Structure 10

1.3.2 WPD - Working Package Description 10

1.3.3 Organisation de l'equipe 13

2 Réalisation **14**

2.1 Mise en place de l'environement de Simulation 14

2.2 Acquisition et traitement des données 15

2.2.1 Récupération et traitement des données 16

2.3 Élaboration et génération de la trajectoire du robot 19

2.3.1 Definition de la trajectoire 19

2.3.2 Control du robot 21

2.4 Présentation des résultats obtenus 24

Summary **27**

Table des figures

1.1 Graphe de la sructure de projet. 10

1.2 Organigramme de l'équipe. 13

2.1 Liste des topics liés à la Kinect 16

2.2 Aperçu des informations publiées sur les différents topics d'intérêt. *Sur la figure 2.2(c), plus un pixel est proche de la Kinect, plus sa teinte est foncée.* 17

2.3 Exemple de sortie du topic `/camera/depth/points` 17

2.4 Résultat de l'application de la politique de filtrage à un objet rond 18

2.5 Résultat de l'application de la politique de filtrage au robot Kuka YouBot 18

2.6 Exemple de représentation d'une carte d'occupation 19

2.7 Les revolutions de la trajectoire en fonction de la hauteur de l'objet 21

2.8 Schéma de calcul de l'orientation de la caméra 22

2.9 Numérisation du robot Kuka YouBot 24

2.10 Tâche de numérisation du robot Kuka YouBot : Trajectoires cible et réalisée du robot 25

2.11 Évolution des erreurs en position et en orientation entre les trajectoires cible et réalisée 26

Introduction

0.1. Objet du document

Ce projet d'intégration a pour objectif de nous initier à l'utilisation de l'environnement *ROS* appliqué à la robotique industrielle. Plus précisément, il consiste à développer une trajectoire pour un bras manipulateur industriel **Yaskawa HC10**, dans le but de numériser un objet à l'aide d'un **capteur 3D**. Conçu sous la forme d'une étude de faisabilité, ce projet se déroule en autonomie et en groupe, et nécessite de documenter chacune des étapes afin de permettre une reproduction fidèle des résultats obtenus. Cela comprend la prise en main des outils et l'implémentation de solutions techniques précises dans un environnement virtuel. Le projet s'articule autour des étapes suivantes :

1. Découverte de l'environnement *ROS* et de l'outil de génération de trajectoire *MoveIt*.
2. Utilisation de l'API *MoveIt* pour la création de trajectoires dans l'espace cartésien.
3. Intégration d'un environnement de simulation (*Gazebo*) ainsi que l'ajout d'un capteur 3D.
4. Exploitation des données du capteur 3D pour ajuster les trajectoires générées.

La réalisation de ce projet se fait sur une machine virtuelle fournie, sous *Ubuntu 20.04*, intégrant *ROS Noetic*, ainsi que des outils tels que *MoveIt*, *ROS Control*, *Python 3* et *Visual Studio Code*.

L'objectif de ce rapport est de retracer le déroulement complet du projet. Dans un premier temps, il présente le contexte, détaillant la compréhension des besoins, leur analyse approfondie, ainsi que l'organisation mise en place pour y répondre. Ensuite, il expose la réalisation des différentes phases du projet, en décrivant de manière précise le processus suivi à chaque étape et le résultat final.

0.2. Documents applicables et de référence

- Cahier des charges sur Moodle :
  https://moodle.univ-tlse3.fr/course/view.php?id=9520
- MoveIt Setup Assistant :
  https://moveit.github.io/moveit_tutorials/doc/setup_assistant/setup_assistant_tutorial.html
- Gazebo Simulation Integration :
  https://moveit.github.io/moveit_tutorials/doc/gazebo_simulation/gazebo_simulation.html
- Move Group Python Interface :
  https://moveit.github.io/moveit_tutorials/doc/move_group_python_interface/move_group_python_interface_tutorial.html
- Mesh Filter with UR5 and Kinect :
  https://moveit.github.io/moveit_tutorials/doc/mesh_filter/mesh_filter_tutorial.html
- Perception Pipeline Tutorial :
  https://moveit.github.io/moveit_tutorials/doc/perception_pipeline/perception_pipeline_tutorial.html

Chapitre 1

Contexte

1.1. Compréhension du besoin

L'objectif principal de ce projet est de permettre au robot HC10 d'exécuter des trajectoires précises dans un environnement simulé pour numériser un objet à l'aide d'un capteur 3D. Cela nécessite la création d'une simulation réaliste du robot dans un espace virtuel, intégrant un capteur simulé. Ce capteur permettra au robot de percevoir son environnement ainsi que l'objet, et de prendre des décisions en temps réel, telles que l'évitement d'obstacles ou la rotation pour capturer l'ensemble des perspectives de l'objet.

Le projet est donc divisé en trois tâches principales :

— **Modélisation du Robot en Simulation :** Il s'agit de créer un modèle simulé du robot capable d'effectuer des mouvements réalistes dans un environnement virtuel.

— **Ajout et Intégration de Capteur :** Ajouter et simuler du capteur, notamment une Kinect, dans Gazebo pour permettre la détection d'objets et la navigation basée sur l'imagerie afin de générer des trajectoires qui permettent l'évitement d'obstacle.

— **Construction et Génération de Trajectoires :** Le robot doit pouvoir interagir avec son environnement virtuel, en adaptant ses mouvements pour visualiser l'objet sous tous les angles et éviter les obstacles.

1.2. Analyse du besoin

1.2.1 Création de la Simulation du Robot :

Objectif

Simuler le robot avec une précision suffisante pour que ses mouvements et interactions avec l'environnement soient réalistes.

Solution Proposée

Utilisation de l'assistant de configuration MoveIt :
Utiliser l'assistant de MoveIt pour générer un package de base contenant les fichiers de configuration nécessaires à la simulation du robot ainsi qu'à la génération de trajectoires via l'API MoveIt.

Modification du package de base :
Réorganiser le fichier de description du robot en format xacro pour une meilleure modularité et maintenabilité. Ajuster également les paramètres de MoveIt et de commande du robot.

Ajout et configuration du capteur 3D (modèle Kinect) :
Intégrer un capteur Kinect dans la simulation, en définissant ses caractéristiques (position, orientation, champ de vision, etc.) et en configurant les topics ROS qui publieront les données capturées par le capteur. Cela permettra de rendre les informations de capteurs disponibles pour d'autres nœuds ROS.

1.2.2 Ajout et intégration du capteur 3D

Objectif

Acquérir et filtrer les données reçues du capteur 3D afin de les utiliser pour générer des trajectoires précises. De plus, il s'agit d'intégrer une carte d'occupation permettant au robot de naviguer en évitant les obstacles tout en scannant l'objet.

Solution Proposée

Utilisation de topics préexistants :
Exploiter les topics contenant les données de la caméra (RGB, profondeur, etc.) pour obtenir des informations complètes sur l'environnement. Ces données seront essentielles pour la perception visuelle du robot.

Filtrage des données :
Une méthode de filtrage est utilisée de sorte à converger uniquement les points pertinents pour la construction de trajectoire.

Utilisation du framework ROS OctoMap :
Intégrer ce plugin de MoveIt permettant de convertir les nuages des points en octomap utilisable pour la génération de trajectoire. Efficace car il permet de filtrer automatiquement les points non pertinents, optimisant ainsi la précision de la carte d'occupation.

1.2.3 Construction et Génération de Trajectoire

Objectif

Le robot doit être capable de recevoir les informations qui viennent du capteur pour tenir en compte les obstacles du terrain et générer une trajectoire tout en évitant ces obstacles.

Solution Proposée

Définition de la Trajectoire :
Générer une trajectoire elliptique autour de l'objet à scanner, en ajustant cette trajectoire en fonction des données fournies par les capteurs par un coefficient pour éviter les collisions.

Envoi de la trajectoire par API :
La transmission des positions calculées à l'outil de contrôle MoveIt, et veiller à ce que la caméra reste orientée vers l'objet pendant le scan. Un script gèrera l'automatisation de ces processus, garantissant un scan précis et sécurisé.

1.3. Organisation

1.3.1 WBS - Working Breakdown Structure

Le schéma suivant (Fig. 1.1) illustre la structure et les étapes du projet, en répartissant clairement les tâches à partir de trois branches principales :

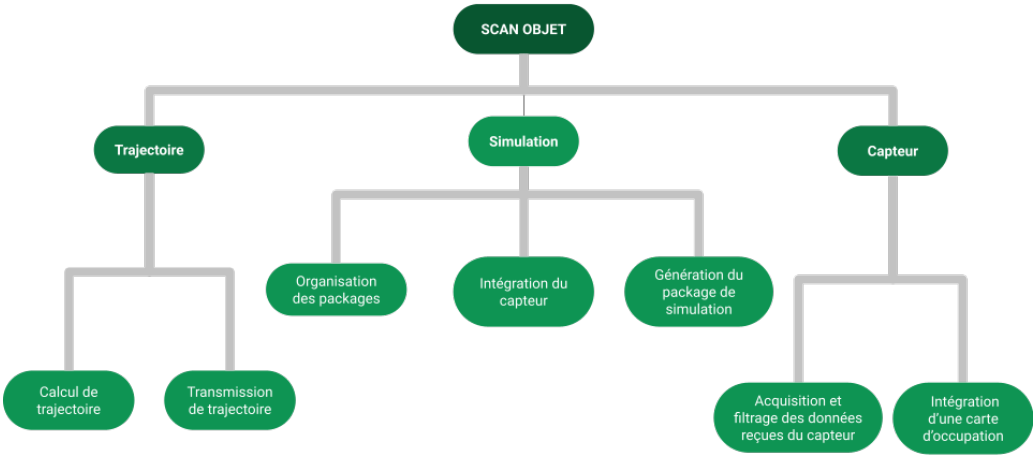


FIGURE 1.1 – Graphe de la sructure de projet.

1.3.2 WPD - Working Package Description

Les tableaux suivants détaillent les différentes tâches du projet, en spécifiant pour chaque tâche : les responsables, le contenu et les résultats. Ils permettent d'assurer une gestion claire et structurée de l'avancement du projet.

Work Package Description	
Nom du Work-Package : Intégration du capteur	**Responsable : Coulibaly/Saidi**
Contenu	1. Création du corps physique de la caméra et de son axe optique ainsi que les joints sur lesquels tout sera fixé.
	2. Ajout du plugin pour permettre la simulation de la caméra dans Gazebo.
Résultats	1. Fichier .xacro de la modélisation du robot avec la caméra fonctionnelle pour utilisation en simulation

Work Package Description	
Nom du Work-Package : Génération du package de simulation	**Responsable : Coulibaly/Ferhani**
Contenu	1. Utilisation de l'assistant MoveIt pour la génération d'un packages pour la simulation et la génération de trajectoire avec MoveIt
Résultats	1. Package de simulation et de génération de trajectoire utilisable pour des tests préliminaires

Work Package Description	
Nom du Work-Package : Organisation des packages	**Responsable : Niato**
Contenu	1. Réorganiser le fichier de description du robot en format xacro
	2. Ajuster les paramètres de MoveIt et de commande du robot.
Résultats	1. Package de simulation fonctionnel et bien organisée

Work Package Description	
Nom du Work-Package : Calcul de trajectoire	**Responsable : Traore/Ghiloufi**
Contenu	1. Génération de la trajectoire elliptique que suivra le robot
	2. Détermination de la position du point le plus élevé
Résultats	1. Fonction retournant l'équation caractéristique de la trajectoire que suivra le robot
	2. Fonction retournant la hauteur du point le plus haut

Work Package Description	
Nom du Work-Package : Transmission de Points	**Responsable : Aloy/Maignan**
Contenu	1. Calcule de l'orientation de la caméra
	2. Envoie des positions de la trajectoire à atteindre au robot
Résultats	1. Le robot se déplace en suivant la trajectoire définie et en évitant les obstacles

Work Package Description	
Nom du Work-Package : Intégration d'une carte d'occupation	**Responsable : Souha/Mouléro**
Contenu	1. Création des fichiers de configuration et de lancement afin de pouvoir lancer octomap_server_node
Résultats	1. Une carte d'occupation d'espace s'affiche dans RViz

Work Package Description	
Nom du Work-Package : Acquisition et filtrage des données reçu du capteur	**Responsable : Mouléro/Nelson**
Contenu	1. Identification le(s) topic(s) d'intérêt
	2. Lecture des données renvoyés sur le(s) topic(s) d'intérêt au bon format
	3. Filtrage des informations superflu du retour des capteurs
Résultats	1. Package de gestion des données capteurs permettant la lecture et le traitement des données capteur
	2. Le nuage de points filtré s'affiche dans RViz

1.3.3 Organisation de l'equipe

la figure 1.2 détail la struturation et l'organisation de l'équipe du projet.

FIGURE 1.2 – Organigramme de l'équipe.

Chapitre 2

Réalisation

Le principal objectif de ce projet est d'exploiter les données fournies par un capteur Kinect, fixé comme organe terminal d'un robot manipulateur Yaskawa HC10, afin de numériser un objet inconnu placé dans l'espace de travail du robot, au sein d'un environnement simulé. À partir des données du capteur, nous devrons générer une série de positions que l'organe terminal du robot devra parcourir. L'environnement de simulation sera développé à l'aide de ROS et Gazebo, tandis que la planification et l'exécution des trajectoires seront réalisées avec le framework ROS MoveIt.

Ce chapitre se composera principalement de quatre sections. Dans la première section, nous aborderons la mise en place de l'environnement de simulation, en présentant l'architecture générale du système et en détaillant les outils et technologies utilisés. La deuxième section traitera du processus de récupération et de traitement des informations issues de la Kinect, afin de permettre la construction d'une trajectoire de numérisation. La troisième partie sera consacrée à la construction et à la réalisation de la trajectoire permettant la numérisation de l'objet. Nous expliquerons la logique employée pour créer la suite de points constituant ces trajectoires, puis nous présenterons l'API MoveIt et son utilisation dans notre projet. Enfin, dans la dernière section, nous illustrerons les résultats obtenus à travers plusieurs exemples visuels, avant de conclure.

2.1. Mise en place de l'environement de Simulation

L'une des parties du projet est l'obtention d'une simulation du robot intégrant le capteur et simulant ces derniers dans des conditions fidèles au monde réel. Dans cette partie, nous décrivons la réalisation de la simulation du robot équipé du capteur Kinect. Cette simulation, réalisée sous Gazebo, nécessite la conception de fichiers URDF pour modéliser le robot et intégrer la Kinect à son extrémité. Elle implique également de simuler les mouvements du robot et de valider la capacité de la Kinect à scanner des objets dans un environnement virtuel.

Dans un premier temps, nous décrivons l'intégration de la caméra Kinect au robot pour permettre son utilisation en simulation.

Dans cette étape de la simulation, un nouveau link (corps) est ajouté pour modéliser physiquement la Kinect. Ce link comprend les dimensions de la caméra afin de permettre un affichage visuel ainsi que la prise en compte de potentielles collisions. Un joint (liaison) est ensuite ajouté pour relier la Kinect au dernier corps du robot. La description de cette liaison inclut la position du joint par rapport à son corps parent, définie en termes de translation et de rotation. De plus, un corps virtuel supplémentaire est créé pour représenter l'axe optique de la caméra, accompagné d'un joint virtuel qui relie cet axe à la Kinect, avec une définition de la position et de l'orientation de l'axe virtuel par rapport au corps physique de la Kinect. Le plugin de Gazebo pour la caméra est également intégré, permettant son utilisation dans l'environnement de simulation. Ce plugin comprend des paramètres liés au fonctionnement de la caméra, tels que les dimensions et le format de l'image de sortie, la portée du capteur, ainsi que d'autres aspects techniques comme les topics de publication des données et les paramètres de distorsion de la caméra. L'ensemble de cette configuration est rédigé dans un fichier xacro, qui sera inclus dans le fichier xacro principal du robot. Après avoir intégré le capteur Kinect, l'étape suivante pour obtenir une simulation fonctionnelle consiste à configurer le solveur utilisé. Pour la simulation nous utilisons le solveur KDL. Lors de la création du fichier URDF utilisé lors de la simulation, le capteur kinect était intégré en tant que "end effector" (organe terminal) mais cet ajout a généré des problèmes : le robot n'était plus considéré comme une chaîne de link.

Nous avons tenté plusieurs actions correctives. La solution finale a été de changer de solveur et, au lieu de définir la Kinect comme organe terminal dans Gazebo, de faire en sorte que le link de la Kinect atteigne une position spécifique.

En conclusion, la création de la simulation sous Gazebo a permis d'intégrer la Kinect au robot. Après avoir modélisé et configuré la caméra, des ajustements ont été nécessaires pour corriger des problèmes de chaîne cinématique. La solution finale a abouti à une simulation fonctionnelle et fidèle. Cette simulation est désormais prête et sera mise à disposition des deux autres équipes pour la suite du projet.

2.2. Acquisition et traitement des données

Le capteur Kinect, intégré dans l'environnement de simulation, est utilisé pour capturer diverses informations représentant la scène environnante. Ces données sont ensuite traitées et exploitées pour générer des positions spatiales, facilitant ainsi la planification des mouvements du bras en prenant en compte les obstacles et l'environnement, grâce à l'API MoveIt.

Pour ce faire, nous avons suivi plusieurs étapes que nous présenterons dans la suite de cette section.

2.2.1 Récupération et traitement des données

L'intégration du capteur dans l'environnement de simulation induit la création de plusieurs topics, permettant de transmettre des informations sur le champ de vision du capteur. La figure 2.1 présente une vue exhaustive de ces topics. Les plus pertinents pour notre application sont ceux encadrés.

```
gexture@g3xTuRe:~$ rostopic list | grep camera
/camera/color/camera_info
/camera/color/image_raw
/camera/color/image_raw/compressed
/camera/color/image_raw/compressed/parameter_descriptions
/camera/color/image_raw/compressed/parameter_updates
/camera/color/image_raw/compressedDepth
/camera/color/image_raw/compressedDepth/parameter_descriptions
/camera/color/image_raw/compressedDepth/parameter_updates
/camera/color/image_raw/theora
/camera/color/image_raw/theora/parameter_descriptions
/camera/color/image_raw/theora/parameter_updates
/camera/depth/camera_info
/camera/depth/image_raw
/camera/depth/points
/camera/depth/points_black
/camera_ir/parameter_descriptions
/camera_ir/parameter_updates
```

FIGURE 2.1 – Liste des topics liés à la Kinect

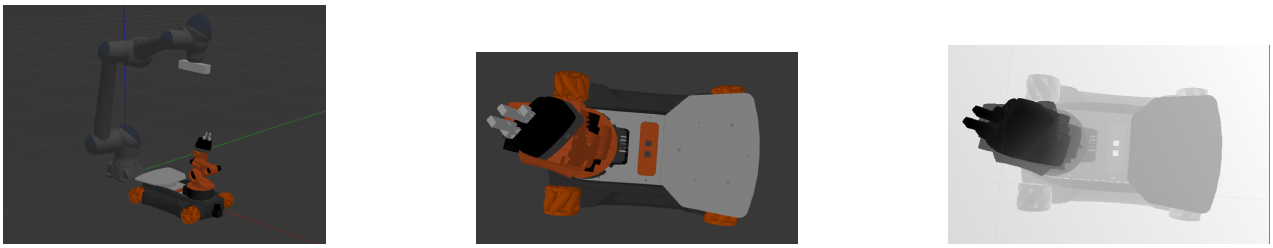
Les topics `/camera/depth/image_raw` et `/camera/color/image_raw` fournissent respectivement des informations de profondeur et des images en couleur. Le topic `/camera/depth/image_raw` publie des images de profondeur brutes, où chaque pixel représente la distance entre la Kinect et les objets présents dans son champ de vision. Cela permet de percevoir la profondeur de la scène et de reconstruire une carte 3D ou un nuage de points. Le topic `/camera/color/image_raw`, quant à lui, diffuse des images couleur brutes capturées par la caméra RGB de la Kinect, qui peuvent être utilisées pour la reconnaissance d'objets ou la superposition avec les données de profondeur.

Le topic `/camera/depth/points`, lui publie un nuage de points (point cloud) en 3D. Ce nuage de points est une représentation tridimensionnelle de la scène capturée par le capteur, où chaque point est défini par des coordonnées spatiales (x, y, z) et une couleur RGB, basées sur les informations de profondeur et de couleur mesurées par la Kinect. C'est ce nuage de points, publié par ce topic, qui revêt le plus d'intérêt pour notre application.

La figure 2.2 présente un aperçu des résultats obtenus sur ces différents topics dans RViz.

Un aperçu d'un nuage 3D obtenu sur le topic `/camera/depth/points` est présenté par la figure 2.3.

Comme le montre la figure 2.3, le nuage de points renvoyé contient non seulement des points correspondant à notre objet, mais aussi des points représentant le sol, ce qui pourrait poser problème



((a)) Le Yaskawa dans l'environnement Gazebo

((b)) Sortie du topic `/camera/color/image_raw`

((c)) Sortie du topic `/camera/depth/image_raw`

FIGURE 2.2 – Aperçu des informations publiées sur les différents topics d'intérêt. *Sur la figure 2.2(c), plus un pixel est proche de la Kinect, plus sa teinte est foncée.*

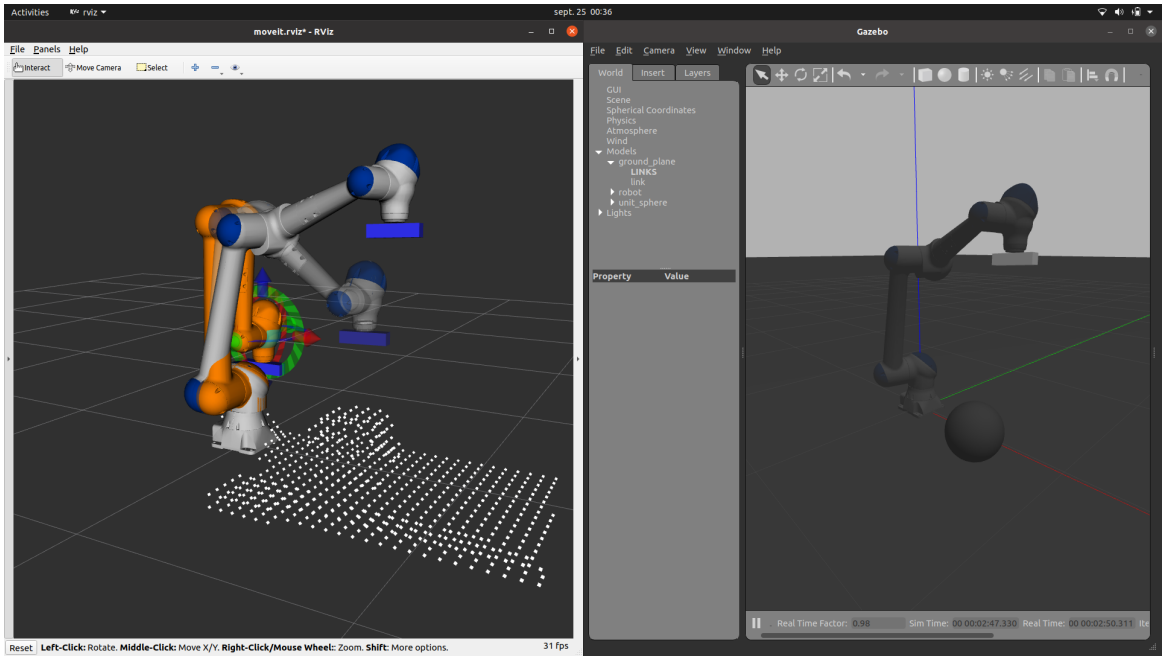
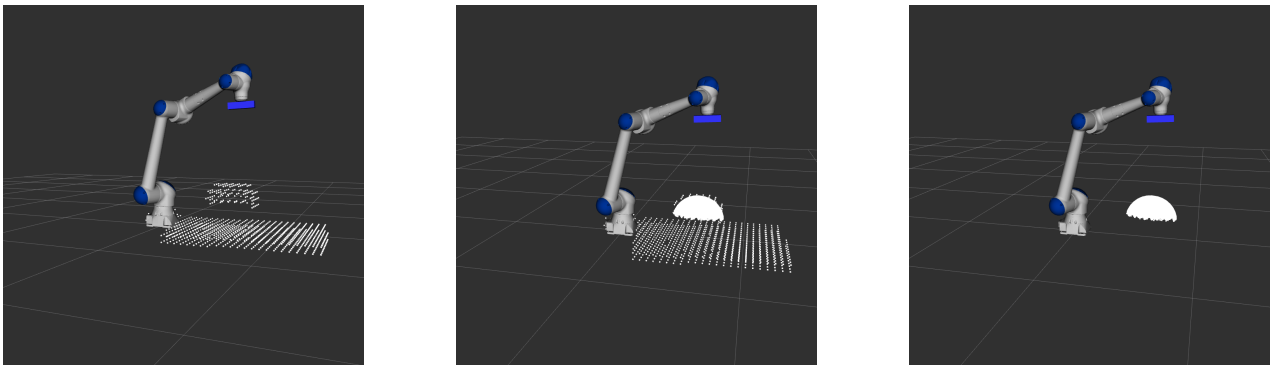


FIGURE 2.3 – Exemple de sortie du topic `/camera/depth/points`

lors du traitement. Par conséquent, une stratégie de filtrage a été mise en place pour exclure les points appartenant au sol.

Cette méthode consiste à effectuer un clustering des points du nuage en calculant la normale en chaque point du nuage de points par rapport au plan formé par celui-ci et le centre du nuage de points. Cela nous permet de déterminer le plan contenant les points du sol : il s'agit de l'ensemble des points majoritaires dont les normales ont une orientation égale ou similaire, et qui sont les plus éloignés du centre du nuage de points. Une fois ce plan identifié, tous les points situés à une certaine distance du plan (selon un seuil $s = 0.196 \times |\texttt{max_z} - \texttt{min_z}|$) sont considérés comme appartenant au sol et sont donc exclus au même titre que tous les points du plan du sol. Le nuage de point restant est en suite renvoyé sur le topic `/camera/depth/points_black`.

Les figures 2.4 et 2.5 montrent quelques exemples des résultats obtenus après traitement.

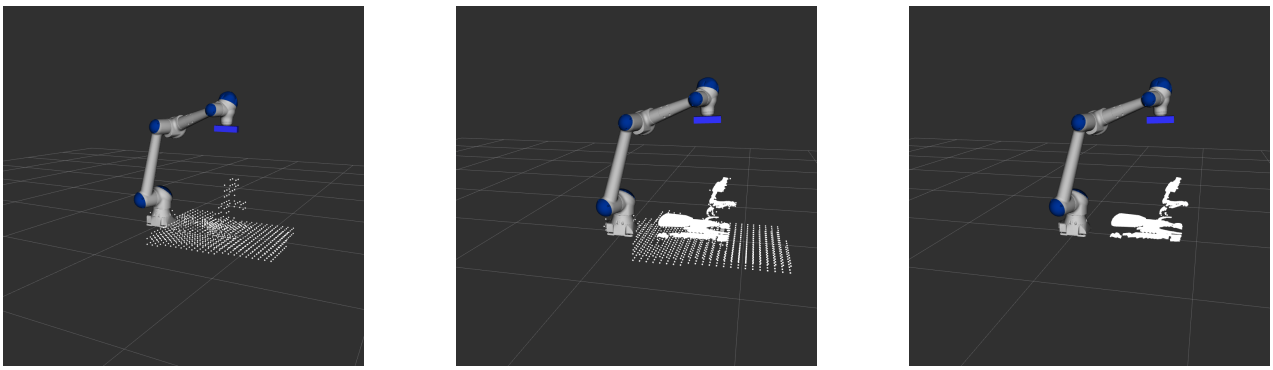


((a)) Nuage de points non filtré

((b)) Superposition des deux nuages de points

((c)) Nuage de points filtré

FIGURE 2.4 – Résultat de l'application de la politique de filtrage à un objet rond



((a)) Nuage de points non filtré

((b)) Superposition des deux nuages de points

((c)) Nuage de points filtré

FIGURE 2.5 – Résultat de l'application de la politique de filtrage au robot Kuka YouBot

Une autre fonction de la Kinect est de permettre la création d'une carte d'occupation d'espace (OctoMap) afin que le framework MoveIt puisse planifier et exécuter des trajectoires sans collision dans l'espace de travail. Pour ce faire, nous avons dû créer un fichier de configuration YAML définissant le plugin à utiliser pour générer la carte d'occupation, le topic du nuage de points à prendre en compte, ainsi que la frame du robot associé au capteur.

Une fois ce fichier créé, nous avons lancé le nœud `octomap_server_node` du paquet `octomap_server` de ROS, en spécifiant la frame de référence pour la génération de la carte d'occupation, d'indiquer la frame associée au capteur, et de remapper le nœud vers le topic fournissant le nuage de points à traiter.

Ces actions nous permettent d'intégrer les informations relatives à la scène dans ROS, facilitant ainsi la planification et l'exécution des trajectoires par MoveIt en tenant compte des obstacles présents dans l'environnement.

La figure 2.6 montre un exemple de carte d'occupation obtenu dans notre environement de simulation.

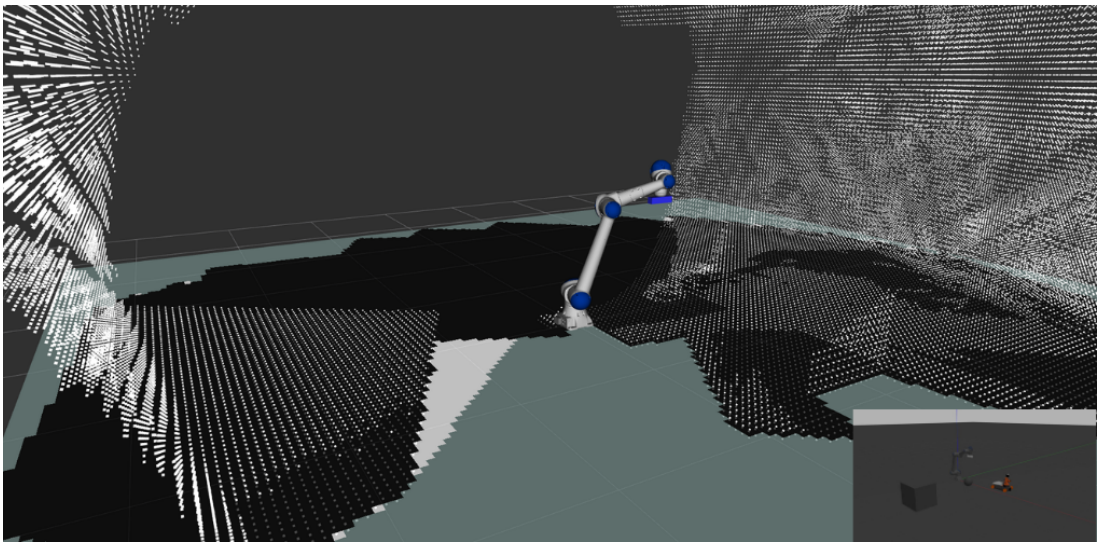


FIGURE 2.6 – Exemple de représentation d'une carte d'occupation

Une fois tous ces élements places, nous pouvons les exploiter afin de définir la suite de positions spaciales que doit suivre le robot durant l'opération de numérisation de l'objet.

2.3. Élaboration et génération de la trajectoire du robot

Dans le but de scanner un objet avec le bras manipulateur Yaskawa HC10, il est nécéssaire de construire la trajectoire du movement du bras. Pour ce faire, il nous faut générer une suite de positions et orientations à suivre par l'organe terminal de notre bras manipulateur.

Cette partie va détailler le choix de la forme et les calculs effectués pour créer la trajectoire ainsi que les outils utilisés pour transmettre ces informations aux bras robots.

2.3.1 Definition de la trajectoire

Cette partie a pour objectif de définir la trajectoire du bras robot, grâce aux données renvoyées par le capteur, afin de pouvoir effectivement scanner l'objet.

Calcul de l'ellipse

Au départ, l'idée était de choisir une trajectoire générique autour de l'objet une fois que sa forme a été déterminée par le capteur. La meilleure façon qu'on a trouvé est de créer une ellipse autour de l'objet et de l'agrandir par un coefficient afin d'éviter toutes les collisions possibles.

Dans un premier temps, nous avons créé une fonction en utilisant les coordonnées des quatre points délimitant le contour de l'objet vu de dessus pour calculer le centre de l'ellipse, ainsi que ses axes principaux (semi-grand axe et semi-petit axe). Nous utilisons ensuite la formule paramétrique de l'ellipse pour générer des points le long du contour.

Afin de récupérer les points nécessaires à la construction de l'ellipse, nous écoutons le topic sur lequel la caméra a publié le nuage de points représentant l'objet vu par la caméra.

Une fois l'ellipse obtenue, nous avons dans un premier temps essayé d'agrandir les axes de 20 unités, cependant nous avons constaté que ce n'était pas très efficace étant donné que le forme de l'ellipse changeait beaucoup dû à cette modification. On a donc décidé d'utiliser à la place un coefficient qui se multipliera aux axes principaux. Cela garantira que la forme d'origine (l'ellipse) soit préservée, mais agrandie uniformément dans toutes les directions.

Nombre de couche

On souhaitait également déterminer le nombre de révolutions autour de l'objet en fonction de sa hauteur, plus l'objet est grand plus le nombre de tours augmente. Deux méthodes ont été explorées pour ajuster dynamiquement le nombre d'ellipses générées en fonction de la hauteur maximale du nuage de points :

1. **Approche basée sur la racine carrée de la hauteur**
   Cette méthode consiste à calculer le nombre de divisions en fonction de la racine carrée de la hauteur maximale Z de l'objet. Cette racine carrée est utilisée pour garantir que le nombre d'ellipses augmente progressivement avec la hauteur, sans que le nombre de divisions ne devienne trop important pour des hauteurs élevées. Le nombre de divisions est ensuite limité à une plage raisonnable grâce à la fonction numpy.clip, permettant de spécifier un minimum et un maximum de divisions. Nous avons conclu que cette approche n'est pas très efficace car par exemple pour un objet de 16cm de hauteur nous allons faire 4 tours et nous pensons que c'est beaucoup pour un objet aussi petit.

2. **Approche basée sur la règle de trois**
   Cette méthode repose sur une simple règle de trois : pour chaque tranche de 10 unités en hauteur, une ellipse est générée. La hauteur maximale du nuage de points est divisée par cette valeur de référence (10), et le résultat est arrondi à l'entier supérieur à l'aide de la fonction math.ceil, afin de s'assurer qu'il y ait au moins une ellipse même si la hauteur n'est pas un multiple exact de 10.

Les deux approches ont été intégrées dans un environnement 3D permettant la visualisation des ellipses générées à différentes hauteurs (Fig.2.7). Chaque méthode présente des avantages en fonction du besoin spécifique : l'approche basée sur la racine carrée est plus progressive, tandis que celle basée sur la règle de trois offre un contrôle plus direct et intuitif.

Afin de s'assurer que le robot ne s'éloigne pas trop de sa cible, on a créé une fonction renvoyant la distance qui sépare la caméra de l'objet, et en fonction de cette distance le robot serait capable de se rapprocher ou de s'éloigner de l'objet.

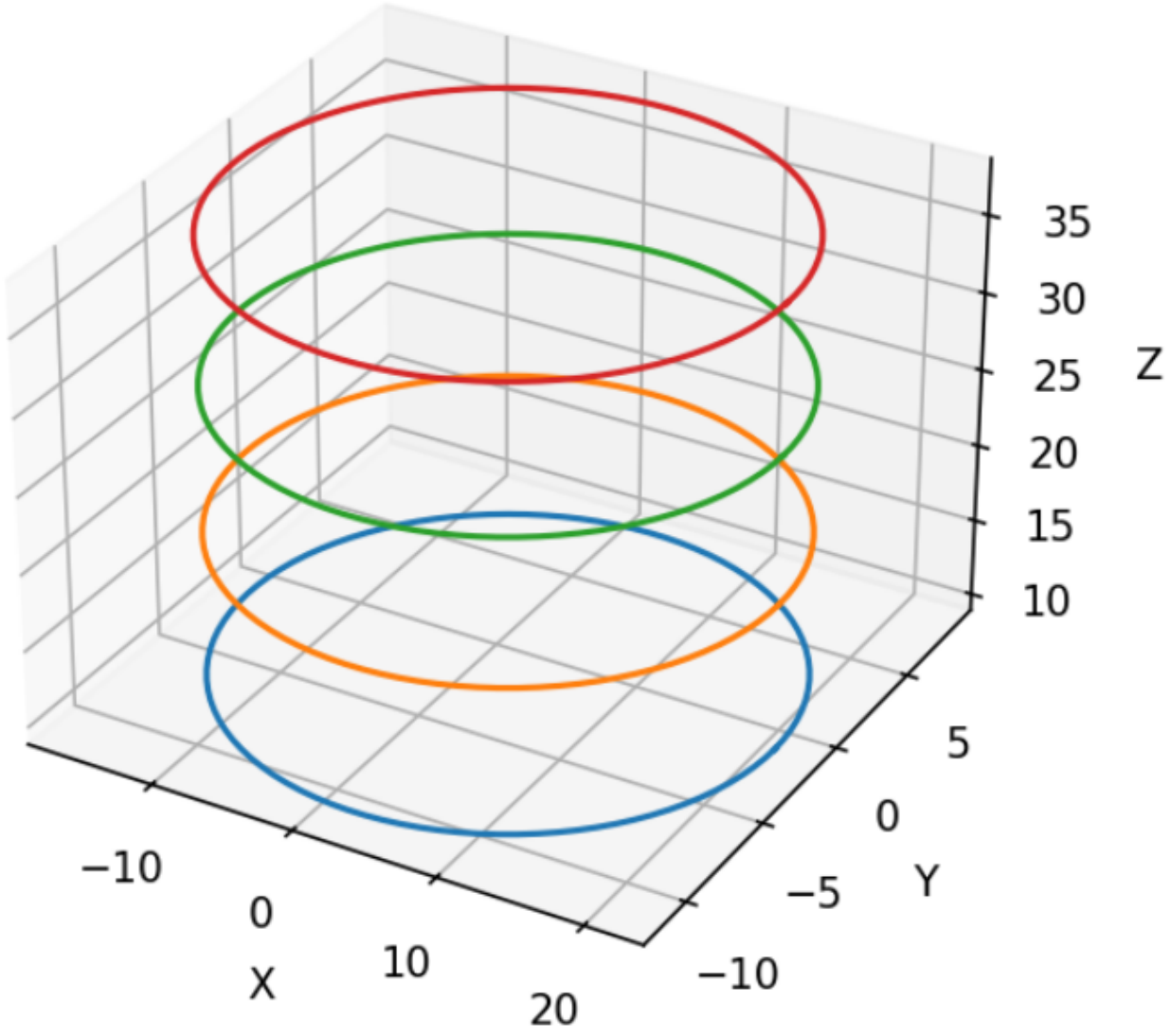


FIGURE 2.7 – Les revolutions de la trajectoire en fonction de la hauteur de l'objet

2.3.2 Control du robot

Lorsque nous avons finalement créé les coordonnées des positions à atteindre par le robot, nous devons ensuite les transmettre à MoveIt qui est un outil permettant de calculer la trajectoire effectuée par le bras pour atteindre cette position.

Connexion à MoveIt

Pour transmettre les positions à atteindre, nous utilisons la librairie moveit_commander en python pour contrôler le bras robot via l'API Moveit. Cela permet de commander le robot soit avec les valeurs de chaque joint (angle de chaque articulation), soit en envoyant une position à atteindre dans le repère monde. Dans notre cas nous créons une trajectoire avec des positions dans le repère monde à atteindre alors nous avons décidé de commander le robot à l'aide d'envoie de positions ce qui permet d'éviter de faire la conversion entre la position dans le monde et les valeurs des joints.

Dans une première version, nous lui envoyons la position à atteindre via un script python qui communique par le biais de l'API MoveIt mais lors de nos tests, une erreur apparaît en nous disant

qu'il manque un end effector (un outil en fin du bras) car à cet instant du projet, la caméra n'était pas encore ajoutée dans la simulation ce qui empêchait Moveit de calculer la trajectoire pour atteindre la position envoyée.

Afin de vérifier que nous pouvons communiquer avec l'API nous avons créé un nouveau script qui contrôle le robot via les joints. Celui-ci nous permet de vérifier la liaison entre notre script et l'API.

Orientation

Suite à cela nous avons décidé de nous concentrer sur l'orientation de la caméra pour qu'elle soit toujours orientée vers l'objet à scanner. Pour cela, on récupère trois coordonnées (Fig.2.8) :

— Celle de la caméra (la position actuelle de l'outil du robot).

— La prochaine position à atteindre.

— Le centre de l'ellipse.

Après avoir obtenu ces trois points, on crée les trois vecteurs suivants :

— Entre la position actuelle et le centre de l'ellipse.

— Entre prochaine position et centre de l'ellipse.

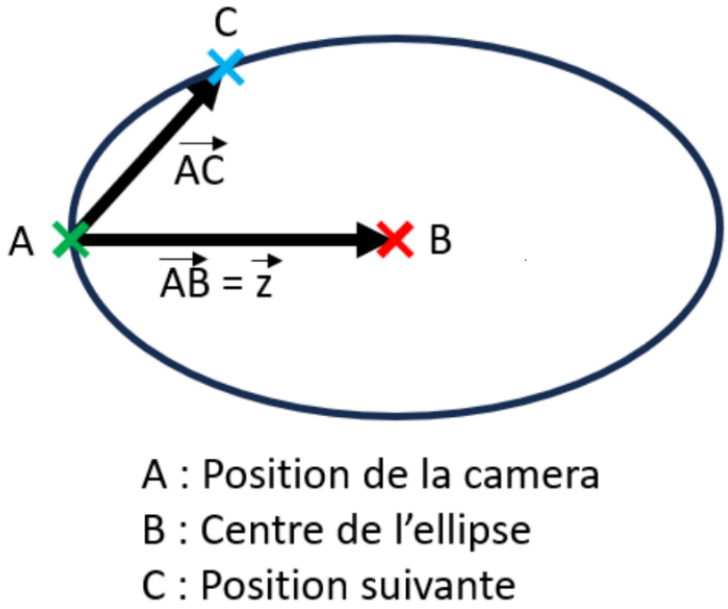


FIGURE 2.8 – Schéma de calcul de l'orientation de la caméra

Ensuite, les vecteurs $\vec{AB}$ et $\vec{AC}$ nous permettent de déterminer le repère de la caméra ($\vec{x}$, $\vec{y}$, $\vec{z}$). En effet, nous avons :

— $\vec{z} = \vec{AB}$

— $\vec{y} = \vec{AB} \cdot \vec{AC}$

— $\vec{x} = \vec{z} \times \vec{y}$

Avec ce repère, nous pouvons calculer la matrice de passage afin d'identifier l'orientation de ce repère dans le repère monde. Il faut ensuite calculer les quaternions (x, y, z, w) associés à cette matrice :

— $w = \frac{1}{2} \cdot \sqrt{1 + R_{0,0} + R_{1,1} + R_{2,2}}$

— $x = \frac{R_{2,1} - R_{1,2}}{4w}$
— $y = \frac{R_{0,2} - R_{2,0}}{4w}$
— $z = \frac{R_{1,0} - R_{0,1}}{4w}$

Intégration trajectoires

Nous avons réalisé un script python qui réunit la création de la trajectoire du robot, le calcul des quaternions pour chaque position et l'envoie des positions à atteindre à MoveIt. Dans ce script, nous commençons par créer les différents points de l'ellipse en récupérant les points transmis sur le topic du capteur. Puis nous exprimons les points du repère caméra au repère monde tout en ordonnant ces points dans l'ordre des positions à atteindre. Pour finir, nous parcourons cette liste de coordonnées x,y,z et calculons les quaternions avant de les transmettre à MoveIt pour que le robot effectue la trajectoire.

Après plusieurs essais, il est apparu que notre approche de génération de trajectoire ne produisait pas les résultats attendus. Les références spatiales et l'orientation du robot étaient incorrectes, entraînant un positionnement erroné de celui-ci dans l'espace après l'exécution des commandes. Il a donc été nécessaire de réviser la méthode utilisée.

L'idée centrale, qui consiste à effectuer une ou plusieurs ellipses autour de l'objet à numériser, reste néanmoins pertinente. Cette révision implique une modification des équations permettant de déterminer la position et l'orientation de chaque point de la trajectoire à suivre par le bras manipulateur.

Soit P un nuage de points résultant de l'observation de l'objet à numériser vu de dessus. Soient C le centre de P, $minZ$ et $maxZ$ respectivement les profondeurs minimales et maximales dans P, ext_x et ext_y les points d'extrémité en X et Y de P, et act_pose la position actuelle de l'effecteur terminal dans le repère monde. Les distances entre C et ext_x ainsi qu'entre C et ext_y permettent de déterminer la largeur de l'objet. $minZ$ et $maxZ$ servent à estimer la hauteur de l'objet.

Notre objectif est de créer une ellipse approximativement centrée sur C, permettant de contourner l'objet tout en assurant que, pour chaque point de la trajectoire, l'orientation de l'effecteur terminal reste dirigée vers C. Cela garantira que l'objet soit maintenu dans le champ de vision de la caméra durant tout le mouvement.

Les positions x, y et z du robot sont calculées en exploitant les équations suivantes :

$$\begin{cases} a &= \text{dist_x} + \epsilon \\ b &= \text{dist_y} + \epsilon \\ x &= C.x + a \cdot \cos(t) \cdot \text{scale} \\ y &= C.y + b \cdot \sin(t) \cdot \text{scale} \\ z &= (1 - t) \cdot (act_pose.z - minZ - \text{threshold}) \end{cases}$$

avec scale $= 1.38$, $\epsilon = 0.12$, $t \in [0, 2\pi]$ et threshold une distance de sécurité.

L'orientation de chaque point (x, y, z) de la trajectoire est calculée en faisant tourner l'organe

terminal du robot d'un angle θ autour de l'axe δ, où :

$$\begin{cases} \theta & = \arccos(Z \cdot d) \\ \delta & = Z \times d \end{cases}$$

avec Z le vecteur définissant la rotation autour de l'axe z de la matrice de rotation entre le repère monde et le repère caméra, et $d = \frac{C-(x,y,z)}{\|C-(x,y,z)\|_2}$.

Pour résumer, la génération d'une trajectoire elliptique garantit une zone de sécurité autour de l'objet, quelle que soit sa forme. De plus, la taille de cette trajectoire s'ajuste automatiquement dès que les contours de l'objet sont complètement visibles par la caméra. Le calcul des quaternions permet ensuite de maintenir l'orientation de la tête du robot en direction du centre de l'objet à tout moment. Après avoir réalisé cette étape ainsi que les autres parties du projet, nous sommes désormais en mesure d'intégrer l'ensemble des résultats pour obtenir et visualiser le résultat final.

2.4. Présentation des résultats obtenus

Cette section présente les résultats obtenus à l'issue de la réalisation du projet.

À la fin du projet, il est désormais possible de voir le robot évoluer dans l'environnement de simulation, d'interagir avec lui, d'ajouter ou de retirer des éléments et objets dans l'espace de simulation, et d'exécuter des tâches de numérisation d'objets.

Pour tester et évaluer les performances de l'application, différents types et formes d'objets ont été utilisés. Parmi eux, on retrouve notamment le robot Kuka YouBot. Dans cette vidéo, on peut observer le robot accomplir sa tâche autour de l'objet. Une fois la tâche réalisée, la figure 2.9 montre le modèle numérisé de l'objet grâce à Octomap.

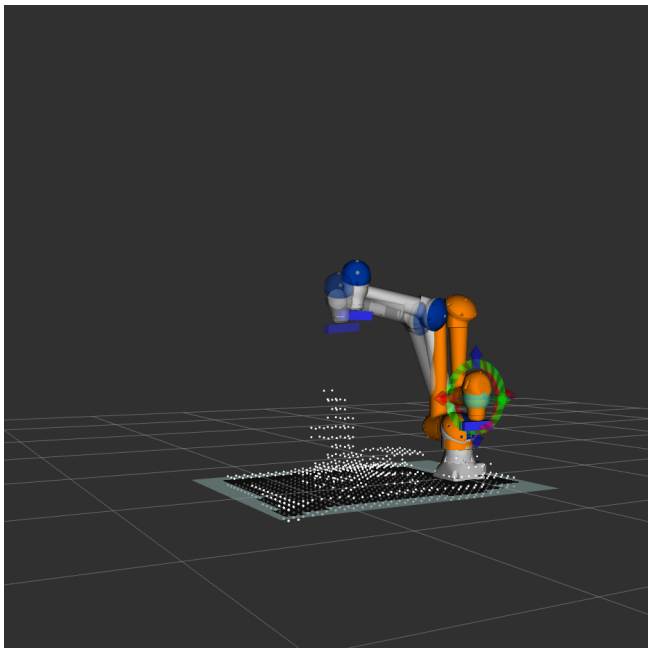
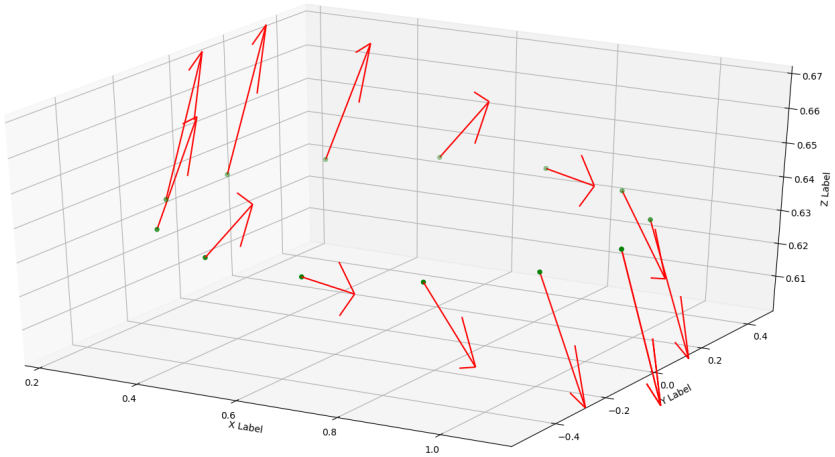


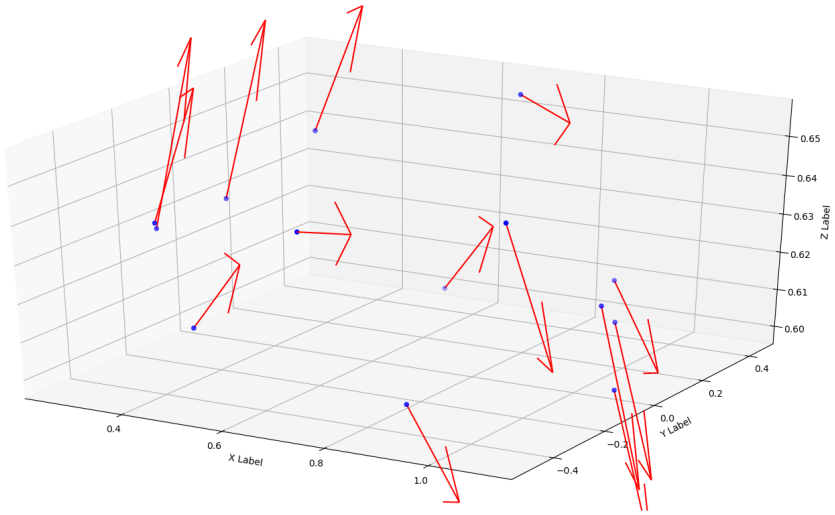
FIGURE 2.9 – Numérisation du robot Kuka YouBot

En termes de performances, nous avons comparé les trajectoires cible et réalisée en déterminant les erreurs en position et en orientation.

La figure 2.10 présente les représentations en 3D des trajectoires cible et réalisée, tandis que la figure 2.11 illustre les erreurs en position et en orientation entre ces deux trajectoires.



((a)) Trajectoire cible



((b)) Trajectoire réalisée

FIGURE 2.10 – Tâche de numérisation du robot Kuka YouBot : Trajectoires cible et réalisée du robot

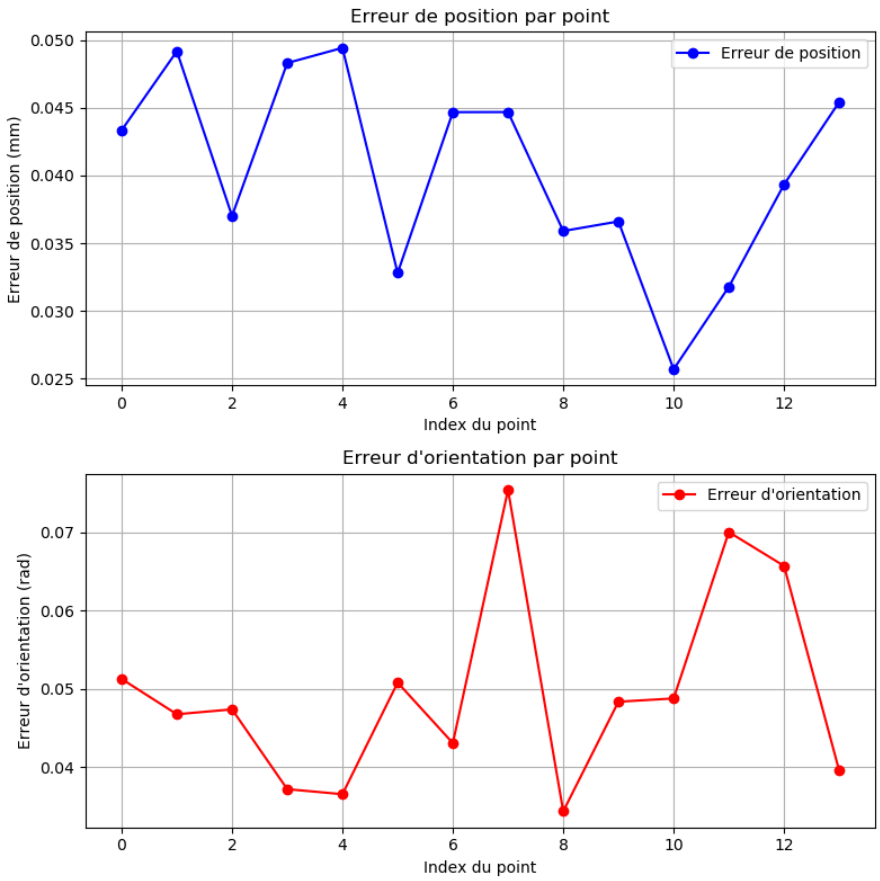


FIGURE 2.11 – Évolution des erreurs en position et en orientation entre les trajectoires cible et réalisée

On observe que les erreurs en position et en orientation sont relativement faibles : l'erreur maximale en position est de l'ordre de $5 \cdot 10^{-2}$ mm, et celle en orientation est de $8 \cdot 10^{-2}$. Cela démontre que la trajectoire est réalisée avec une précision satisfaisante.

Conclusion

En conclusion, ce projet nous a permis de nous familiariser avec l'environnement *ROS* et son application à la robotique industrielle, tout en mettant en pratique nos connaissances en simulation et en contrôle de trajectoire. Le développement de trajectoires pour le bras manipulateur **Yaskawa HC10**, combiné à l'utilisation d'un **capteur 3D**, nous a permis d'aborder des défis techniques interessantes, notamment l'intégration de données de capteur en temps réel pour ajuster les mouvements du robot.

La collaboration en groupe et la documentation détaillée de chaque étape, même si difficile à réaliser, ont été des éléments essentiels à la réussite du projet, garantissant ainsi une reproductibilité des résultats. Les différentes phases, allant de la découverte de *ROS* et *MoveIt* à l'intégration de l'environnement de simulation et du capteur, ont été menées de manière progressive et réfléchie, avec pour objectif de simuler avec précision un processus de numérisation dans un cadre virtuel.

Ce projet a non seulement renforcé notre maîtrise des outils robotiques dans un environnement simulé, mais il a également mis en lumière l'importance de la planification des trajectoires dans les systèmes industriels. Les résultats obtenus ouvrent la voie à des améliorations futures, notamment en ce qui concerne l'optimisation des trajectoires et l'intégration de systèmes plus complexes.

En somme, cette expérience a posé des bases solides pour de futurs travaux dans le domaine de la robotique, et constitue une étape clé dans notre apprentissage de la manipulation robotique via l'écosystème *ROS*.